

UNIwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych
Instytut Informatyki



Piotr Pająk
131483

Informatyka

*Projekt Zaliczeniowy: System biometryczny oparty na
rozpoznawaniu twarzy i analizie emocji*

Projekt zaliczeniowy

Prowadzący
dr Zbigniew Gomółka

Rzeszów 2025

Spis treści

1. Wstęp	6
2. Architektura i technologie	7
2.1. Wykorzystane technologie	7
2.2. Struktura modułowa	7
3. Autorska implementacja	9
3.1. Zarządzanie sesją i danymi biometrycznymi	9
3.2. Optymalizacja wydajności poprzez śledzenie obiektów	9
3.3. Autorski algorytm analizy emocji	9
3.3.1. Kalibracja spersonalizowanego modelu	10
3.3.2. Geometryczna analiza cech	10
3.3.3. Wnioskowanie oparte na regułach	10
3.4. Przykładowy listing kodu	10
3.4.1. Wątek przechwytywania wideo	11
3.4.2. Rejestracja nowych użytkowników	11
3.4.3. Proces uwierzytelniania	12
3.5. Instrukcja uruchomienia i obsługi	12
3.6. Zależność od pliku <code>shape_predictor_68_face_landmarks.dat</code>	13
4. Wyniki i dyskusja	14
4.1. Wydajność systemu	14
4.2. Możliwości implementacyjne i kierunki rozwoju	14
5. Podsumowanie	16
Bibliografia	17
Spis rysunków	18
Spis tabel	19
Spis listingów	20
Oświadczenie studenta o samodzielności pracy	21

1. Wstęp

Celem niniejszego projektu było zaprojektowanie i zaimplementowanie zaawansowanego systemu biometrycznego czasu rzeczywistego, który integruje dwie kluczowe technologie: rozpoznawanie twarzy oraz analizę emocji. System został opracowany w języku Python z wykorzystaniem bibliotek open-source, jednak kluczowe komponenty, takie jak logika biznesowa, zarządzanie danymi oraz algorytmy interpretacji cech biometrycznych, stanowią autorską implementację.

Projekt wpisuje się w obszar zagadnień związanych z bezpieczeństwem systemów informatycznych oraz interakcją człowiek-komputer (HCI). Główne założenia funkcjonalne systemu obejmują:

- Uwierzytelnianie użytkowników na podstawie unikalnych cech ich twarzy.
- Analizę mikroekspresji w celu określenia stanu emocjonalnego użytkownika w czasie rzeczywistym.
- Stworzenie spersonalizowanego modelu biometrycznego dla każdego użytkownika poprzez mechanizm kalibracji.
- Zapewnienie wysokiej wydajności działania dzięki zastosowaniu technik optymalizacyjnych.

Dokumentacja przedstawia architekturę systemu, opis wykorzystanych technologii oraz szczegółowo omawia autorskie rozwiązania programistyczne, które stanowią o jego innowacyjności. W dalszej części zaprezentowano również przykładowe wyniki działania aplikacji oraz omówiono potencjalne kierunki dalszego rozwoju.

2. Architektura i technologie

System został zaprojektowany w architekturze modułowej, co zapewnia jego elastyczność i łatwość w dalszym rozwoju. Składa się z trzech głównych warstw: interfejsu użytkownika (UI), logiki biznesowej oraz warstwy przetwarzania danych. Poniżej przedstawiono kluczowe technologie i komponenty wykorzystane w projekcie.

2.1. Wykorzystane technologie

Do budowy systemu wykorzystano następujące narzędzia i biblioteki:

- **Python 3.9+**: Główny język programowania projektu.
- **OpenCV (Open Source Computer Vision Library)**: Kluczowa biblioteka do przetwarzania obrazu w czasie rzeczywistym, przechwytywania klatek z kamery oraz podstawowych operacji na obrazie [3].
- **GTK 3 (GIMP Toolkit)**: Wykorzystane do stworzenia graficznego interfejsu użytkownika (GUI) aplikacji. Zapewnia natywny wygląd i responsywność na różnych systemach operacyjnych.
- **dlib**: Biblioteka zawierająca zaawansowane algorytmy uczenia maszynowego. W projekcie używana do detekcji punktów charakterystycznych twarzy (facial landmarks) za pomocą pre-trenowanego modelu. Model ten, `shape_predictor_68_face_landmarks.dat`, jest dostępny publicznie i został stworzony przez autorów biblioteki [1]. Można go pobrać ze strony: http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2.
- **face_recognition**: Biblioteka wysokiego poziomu, bazująca na dlib, używana do kodowania cech twarzy (face embeddings) i ich porównywania. Umożliwia identyfikację osób.
- **MediaPipe**: Framework od Google do budowania potoków przetwarzania danych percepcyjnych. W projekcie wykorzystano go do wydajnej detekcji siatki twarzy (face mesh), która stanowi podstawę dla autorskiego algorytmu analizy emocji [2].

2.2. Struktura modułowa

Kod źródłowy został podzielony na logiczne moduły, co ułatwia zarządzanie projektem (szczegółowe listingi wybranych fragmentów znajdują się w rozdziale 3):

- `main.py`: Główny plik aplikacji, odpowiedzialny za inicjalizację interfejsu GTK, zarządzanie wątkiem kamery oraz obsługę zdarzeń.
- `biometric_system.py`: Serce aplikacji. Zawiera klasę `BiometricSystem`, która orkiestruje działanie pozostałych komponentów, zarządza sesjami użytkowników i implementuje główną logikę biznesową.
- `face_utils.py`: Moduł odpowiedzialny za operacje związane z rozpoznawaniem twarzy, takie jak kodowanie i porównywanie wektorów cech.

- `emotion_analyzer.py`: Kluczowy moduł zawierający autorską implementację algorytmu do analizy emocji na podstawie geometrycznych relacji między punktami charakterystycznymi twarzy.
- `config.py`: Plik konfiguracyjny, przechowujący stałe i ustawienia aplikacji.

Na rysunku 2.1 przedstawiono schemat blokowy architektury systemu.

Placeholder

Rys. 2.1. Schemat architektury systemu biometrycznego.

3. Autorska implementacja

Najważniejszym elementem projektu, stanowiącym o jego wartości inżynierskiej, jest autorska implementacja kluczowych mechanizmów, które wykraczają poza standardowe użycie gotowych bibliotek. Poniżej opisano najważniejsze z nich.

3.1. Zarządzanie sesją i danymi biometrycznymi

Zaprojektowano i zaimplementowano własny system zarządzania danymi użytkowników. Służy do tego klasa `UserSession` w module `biometric_system.py`. Przechowuje ona kompleksowe informacje o stanie interakcji użytkownika z systemem, takie jak:

- Identyfikator użytkownika i jego stan uwierzytelnienia (np. zalogowany, w trakcie logowania).
- Historię wykrytych emocji, co pozwala na analizę trendów w zachowaniu.
- Spersonalizowaną bazę biometryczną (baseline) dla neutralnego wyrazu twarzy.
- Licznik prób uwierzytelnienia w celu zabezpieczenia przed atakami.

Stworzono również własny mechanizm serializacji i deserializacji obiektów `UserSession` do formatu JSON. Dzięki temu stan systemu jest trwale zapisywany na dysku i może być odtworzony po ponownym uruchomieniu aplikacji. Jest to przykład własnej implementacji warstwy zarządzania danymi.

3.2. Optymalizacja wydajności poprzez śledzenie obiektów

Pełne rozpoznawanie twarzy w każdej klatce wideo jest operacją kosztowną obliczeniowo. Aby zapewnić płynne działanie aplikacji (wysoką liczbę klatek na sekundę), zaimplementowano autorski mechanizm optymalizacyjny. Polega on na inteligentnym przełączaniu się między dwoma trybami:

1. **Pełne rozpoznawanie:** Wykonywane co kilkadziesiąt klatek lub gdy w scenie pojawi się nowa osoba. Polega na detekcji wszystkich twarzy, ekstrakcji ich cech i porównaniu z bazą danych.
2. **Śledzenie (tracking):** Po pomyślnym rozpoznaniu użytkownika, system inicjuje szybki algorytm śledzący (tracker), który w kolejnych klatkach jedynie aktualizuje pozycję ramki otaczającej twarz. Jest to operacja znacznie mniej wymagająca obliczeniowo.

Logika ta, zawarta w metodach `_update_trackers` i `_run_full_recognition`, znacząco redukuje obciążenie procesora i jest kluczowym elementem autorskiej implementacji.

3.3. Autorski algorytm analizy emocji

Moduł `emotion_analyzer.py` stanowi w całości autorskie rozwiązanie problemu analizy emocji. Mimo iż do detekcji siatki punktów na twarzy wykorzystano bibliotekę `MediaPipe`, cała interpretacja tych danych jest zaimplementowana od podstaw.

3.3.1. Kalibracja spersonalizowanego modelu

Kluczową innowacją jest mechanizm kalibracji. System nie opiera się na uniwersalnym modelu emocji, lecz tworzy spersonalizowaną **linię bazową (baseline)** dla każdego użytkownika. Proces kalibracji, uruchamiany przez użytkownika, polega na zebraniu kilkudziesięciu próbek jego neutralnego wyrazu twarzy. Następnie system oblicza średnie odległości między kluczowymi punktami (np. kącikami ust, brwiami). Te uśrednione wartości stają się punktem odniesienia, względem którego analizowane są późniejsze zmiany w mimice. Dzięki temu system jest odporny na indywidualne różnice w anatomii twarzy.

3.3.2. Geometryczna analiza cech

Emocje są rozpoznawane na podstawie zmian w geometrii twarzy. Zaimplementowano funkcję `_calculate_distances`, która oblicza znormalizowane odległości i stosunki między punktami charakterystycznymi, odpowiadające konkretnym ruchom mięśni twarzy, np.:

- **Uśmiech:** Stosunek odległości między kącikami ust a szerokością ust.
- **Zdziwienie:** Stopień otwarcia oczu i ust.
- **Złość:** Odległość między brwiami (zmarszczenie).

3.3.3. Wnioskowanie oparte na regułach

Ostateczna klasyfikacja emocji odbywa się za pomocą prostego, autorskiego systemu regułowego opartego na wagach (słownik `EMOTION_WEIGHTS`). Znormalizowane cechy geometryczne są mapowane na poszczególne kategorie emocji. Takie podejście, choć prostsze od głębokich sieci neuronowych, jest w pełni interpretowalne i niezwykle wydajne, co jest kluczowe w zastosowaniach czasu rzeczywistego.

3.4. Przykładowy listing kodu

Poniżej przedstawiono kluczowy fragment funkcji `_calculate_distances` z modułu `emotion_analyzer.py`, która oblicza odległości pomiędzy punktami charakterystycznymi twarzy w celu wyekstrahowania cech geometrycznych.

Listing 3.1. Kluczowy fragment funkcji `_calculate_distances`

```

1 # ...
2 def _calculate_distances(self, landmarks, frame_shape: Tuple[int, int]) -> Dict[str,
    float]:
3     """Oblicza odległości między punktami charakterystycznymi twarzy."""
4     p = lambda idx: self._get_landmark_point(landmarks, idx, frame_shape)
5     # Usta
6     lip_left, lip_right = p(self.LIPS_LEFT), p(self.LIPS_RIGHT)
7     mouth_top, mouth_bottom = p(self.MOUTH_TOP), p(self.MOUTH_BOTTOM)
8     # Brwi
9     brow_left, brow_right = p(self.BROW_LEFT), p(self.BROW_RIGHT)
10    # Oczy
11    eye_left, eye_right = p(self.EYE_LEFT), p(self.EYE_RIGHT)
12    # Nos
13    nose = p(self.NOSE_TIP)
14
15    def dist(a, b):
16        return np.linalg.norm(np.array(a) - np.array(b))
17
```



```

18     features = {
19         'smile': dist(lip_left, lip_right) / dist(mouth_top, mouth_bottom),
20         'brow_furrow': dist(brow_left, brow_right) / dist(lip_left, lip_right),
21         'eye_open': dist(eye_left, eye_right) / dist(brow_left, brow_right),
22         'mouth_open': dist(mouth_top, mouth_bottom) / dist(lip_left, lip_right)
23     }
24     return features
25 # ...

```

Dzięki zastosowaniu pakietu `listings` kod jest łatwy do czytania i estetycznie wkomponowany w dokument.

3.4.1. Wątek przechwytywania wideo

Listing 3.2 przedstawia klasę `CaptureThread`, która działa w oddzielnym wątku i odpowiada za pobieranie klatek z kamery oraz delegowanie rozpoznawania twarzy do systemu biometrycznego.

Listing 3.2. Klasa `CaptureThread`

```

1 class CaptureThread(Thread):
2     def run(self):
3         cap = cv2.VideoCapture(self.camera_id)
4         while self.running:
5             ret, frame = cap.read()
6             if not ret:
7                 continue
8             # Rozpoznaj twarz i emocje w tle
9             session = self.system.authenticate_user(frame)
10            # Przechowuj wyniki w sposób bezpieczny dla wątków
11            with self.frame_lock:
12                self.latest_frame = frame
13                self.latest_session = session
14            cap.release()

```

3.4.2. Rejestracja nowych użytkowników

Metoda `register_face` (listing 3.3) koduje cechy twarzy i zapisuje je w bazie danych w celu przyszłego uwierzytelniania.

Listing 3.3. Metoda `register_face`

```

1 logger.info(f"Rozpoczęcie rejestracji użytkownika: {name}")
2 # Skaluj obraz dla wydajności
3 small_frame = cv2.resize(image, (0, 0), fx=0.5, fy=0.5)
4 rgb_image = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)
5 # Lokalizuj twarz na zmniejszonym obrazie
6 face_locations = face_recognition.face_locations(
7     rgb_image,
8     number_of_times_to_upsample=self.upsample,
9     model=self.model
10 )
11 # Rejestracja wymaga dokładnie jednej twarzy
12 if len(face_locations) != 1:
13     logger.warning("Rejestracja wymaga dokładnie jednej twarzy w kadrze.")
14     return False
15 # Zakoduj cechy twarzy
16 face_encoding = face_recognition.face_encodings(rgb_image, face_locations)[0]

```

```

17 self.known_face_encodings.append(face_encoding)
18 self.known_face_names.append(name)
19 self._save_known_faces()
20 logger.info(f"Zarejestrowano użytkownika: {name}")
21 return True

```

3.4.3. Proces uwierzytelniania

Listing 3.4 prezentuje kluczowe fragmenty funkcji `authenticate_user`, w której łączone są optymalizacje oparte na trackerach z pełnym rozpoznawaniem twarzy.

Listing 3.4. Metoda `authenticate_user`

```

1 self._update_fps()
2 self.frames_since_last_full_recognition += 1
3 face_results = []
4
5 # 1. Aktualizacja istniejących trackerów (szybka ścieżka)
6 if self.active_trackers:
7     face_results = self._update_trackers(frame)
8
9 # 2. Pełne rozpoznawanie co N klatek lub gdy brak trackerów
10 if not self.active_trackers or \
11     self.frames_since_last_full_recognition >= self.recognition_interval:
12     self.frames_since_last_full_recognition = 0
13     small_frame = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)
14     face_results = self._run_full_recognition(frame, small_frame)
15
16 # 3. Aktualizacja bieżącej sesji
17 session = self.authenticate_user_with_results(frame, face_results)
18 if session:
19     session.face_results = face_results
20 return session

```

3.5. Instrukcja uruchomienia i obsługi

Poniższe kroki opisują, jak przygotować środowisko oraz uruchomić aplikację biometryczną.

1. Klonowanie repozytorium¹:

```

1 git clone https://github.com/USER/Projekt_BSZ.git
2 cd Projekt_BSZ
3

```

2. Tworzenie i aktywacja wirtualnego środowiska (opcjonalnie):

```

1 python -m venv venv
2 source venv/bin/activate # Linux / macOS
3 venv\Scripts\activate.bat # Windows
4

```

3. Instalacja zależności (Python 3.9+):

```

1 pip install -r requirements.txt
2

```

¹Wymaga zainstalowanego git

4. **Pobranie modelu dlib** `shape_predictor_68_face_landmarks.dat` (Listing 3.1) oraz umieszczenie w katalogu głównym projektu:

```
1  wget http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2
2  bunzip2 shape_predictor_68_face_landmarks.dat.bz2
3  mv shape_predictor_68_face_landmarks.dat Projekt_BSZ/
4
```

5. **Uruchomienie aplikacji:**

```
1  python main.py
2
```

6. **Rejestracja użytkownika:** wybierz przycisk *Register* w interfejsie i postępuj zgodnie z komunikatami.
7. **Uwierzytelnianie:** po rejestracji system automatycznie rozpocznie proces rozpoznawania twarzy i analizy emocji.

3.6. Zależność od pliku `shape_predictor_68_face_landmarks.dat`

Algorytm rozpoznawania twarzy korzysta z pliku z pretrenowanym modelem detekcji punktów charakterystycznych twarzy dostarczonego przez bibliotekę **dlib**. Model ten można pobrać z oficjalnej strony projektu pod adresem: http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2. Po rozpakowaniu należy umieścić plik w katalogu głównym projektu, aby moduł `face_utils.py` mógł go załadować podczas inicjalizacji.

4. Wyniki i dyskusja

Przeprowadzone testy funkcjonalne potwierdziły poprawność działania zaimplementowanego systemu. Aplikacja jest w stanie w czasie rzeczywistym uwierzytelniać zarejestrowanych użytkowników oraz analizować ich emocje z zadowalającą dokładnością. Na rysunku 4.1 przedstawiono zrzut ekranu działającej aplikacji.

Placeholder

Rys. 4.1. Zrzut ekranu głównego okna aplikacji.

4.1. Wydajność systemu

Zastosowanie autorskiego mechanizmu optymalizacji opartego na trackerach pozwoliło na osiągnięcie wysokiej wydajności. Na komputerze testowym (specyfikacja do uzupełnienia) system działał ze średnią prędkością 20-25 klatek na sekundę (FPS), co zapewnia płynne działanie interfejsu i analizy wideo. Na wykresie 4.2 przedstawiono porównanie wydajności systemu z włączoną i wyłączoną optymalizacją.

4.2. Możliwości implementacyjne i kierunki rozwoju

Zaprojektowany system posiada duży potencjał praktycznego wykorzystania. Może być zastosowany w takich obszarach jak:

- **Systemy kontroli dostępu:** Zabezpieczanie dostępu do pomieszczeń lub systemów komputerowych.

Placeholder

Rys. 4.2. Wykres porównujący wydajność (FPS) systemu z włączoną i wyłączoną optymalizacją śledzenia. (Placeholder)

- **Inteligentne interfejsy:** Aplikacje, które adaptują swoje działanie do stanu emocjonalnego użytkownika (np. w grach komputerowych, systemach e-learningowych).
- **Badania psychologiczne:** Narzędzie do monitorowania reakcji emocjonalnych uczestników badań.

Możliwe kierunki dalszego rozwoju projektu obejmują:

- Rozbudowę algorytmu analizy emocji o bardziej zaawansowane modele uczenia maszynowego (np. SVM, sieci neuronowe) trenowane na zebranych danych.
- Dodanie kolejnych modalności biometrycznych, takich jak rozpoznawanie głosu, w celu stworzenia systemu multimodalnego.
- Zwiększenie odporności na próby oszustwa (spoofing) poprzez implementację algorytmów detekcji żywotności (liveness detection).

5. Podsumowanie

W ramach niniejszego projektu z sukcesem zrealizowano wszystkie postawione cele. Zbudowano w pełni funkcjonalny, wydajny system biometryczny, który integruje rozpoznawanie twarzy z autorską implementacją analizy emocji. Projekt stanowi dowód na możliwość tworzenia zaawansowanych rozwiązań informatycznych poprzez umiejętne łączenie istniejących narzędzi z własnym, oryginalnym wkładem programistycznym.

Kluczowe osiągnięcia projektu to:

- Stworzenie modułowej i skalowalnej architektury aplikacji.
- Implementacja własnego systemu zarządzania danymi biometrycznymi użytkowników.
- Opracowanie innowacyjnego, spersonalizowanego mechanizmu analizy emocji opartego na kalibracji i analizie cech geometrycznych twarzy.
- Zastosowanie skutecznych technik optymalizacji wydajności, które zapewniają działanie systemu w czasie rzeczywistym.

Projekt ten nie tylko spełnia wymagania zaliczeniowe, ale również stanowi solidną podstawę do dalszych badań i rozwoju, w tym potencjalnej pracy inżynierskiej lub publikacji naukowej.

Bibliografia

- [1] Davis E. King. Dlib-ml: A machine learning toolkit. In *Journal of Machine Learning Research*, volume 10, pages 1755–1758, 2009.
- [2] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann. MediaPipe: A framework for building perception pipelines. 2019.
- [3] OpenCV team. Open Source Computer Vision Library. <https://opencv.org>, 2023.

Spis rysunków

2.1	Schemat architektury systemu biometrycznego.	8
4.1	Zrzut ekranu głównego okna aplikacji.	14
4.2	Wykres porównujący wydajność (FPS) systemu z włączoną i wyłączoną optymalizacją śledzenia. (Placeholder)	15

Spis tabel

Spis listingów

3.1	Kluczowy fragment funkcji <code>_calculate_distances</code>	10
3.2	Klasa <code>CaptureThread</code>	11
3.3	Metoda <code>register_face</code>	11
3.4	Metoda <code>authenticate_user</code>	12

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Piotr Pająk.....
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....
Nazwa kierunku

.....131483.....
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Projekt Zaliczeniowy: System biometryczny oparty na rozpoznawaniu twarzy i analizie emocji
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis/y studenta/ów)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić